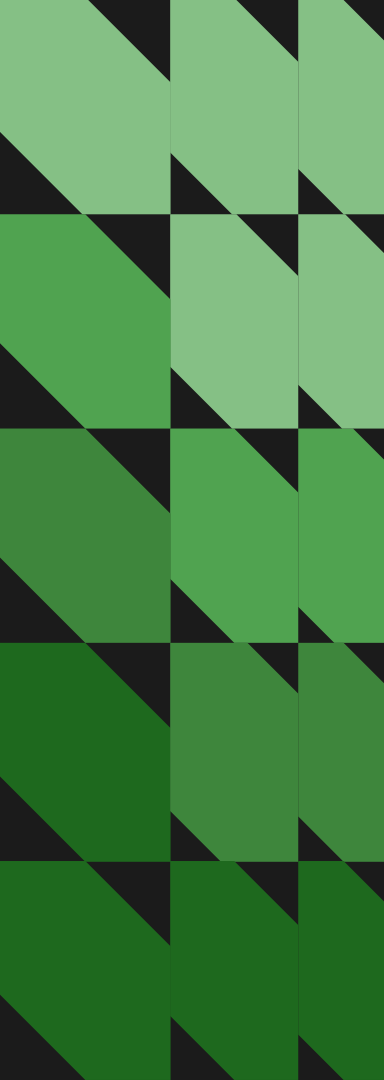




Padrões de projeto

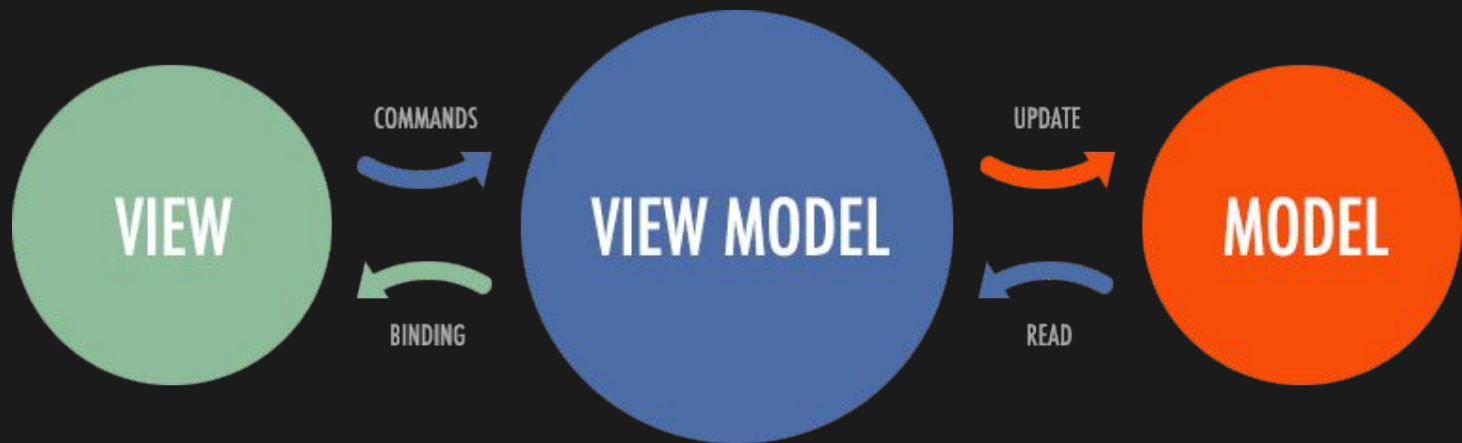
Instrutor: Ralf Lima



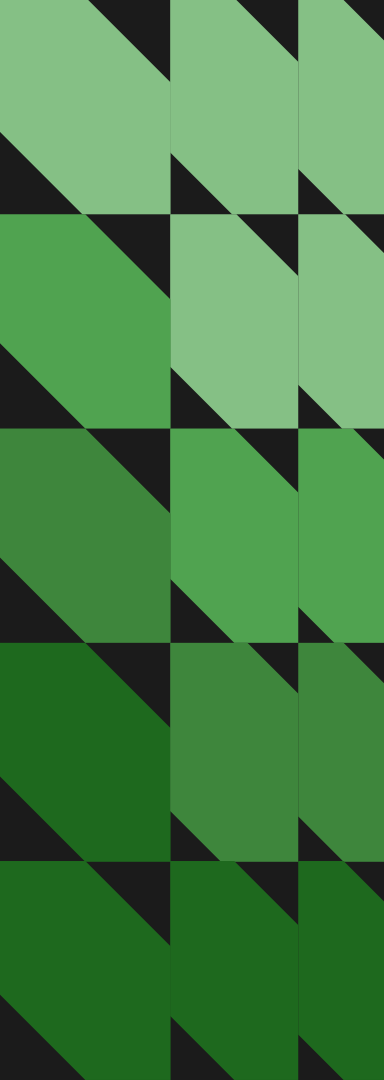
O que é MVVM

MVVM ou *Model-View-ViewModel*, é um padrão de arquitetura de software que separa a interface do usuário (View) da lógica de negócios e dados (Model) utilizando uma camada intermediária (ViewModel).

Como funciona



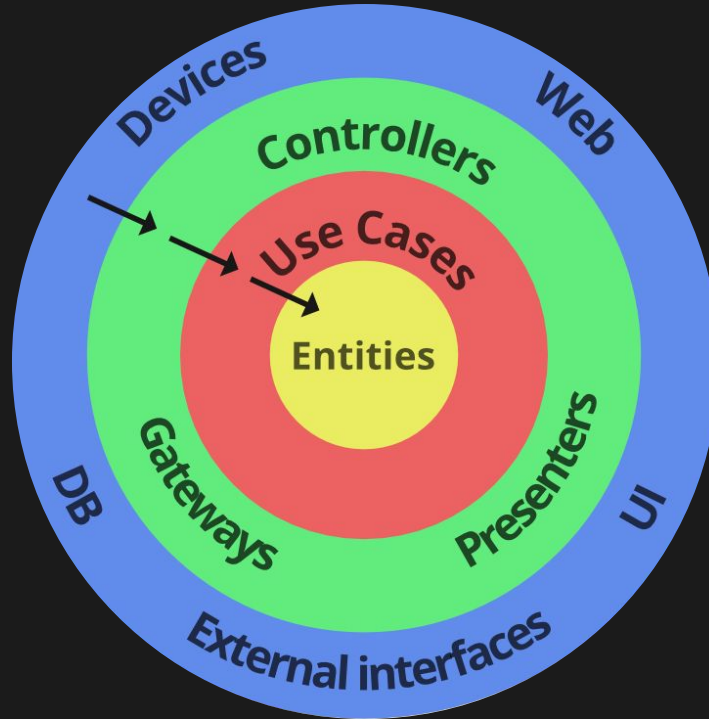
Fonte: <https://github.com/topics/android-mvvm-architecture>

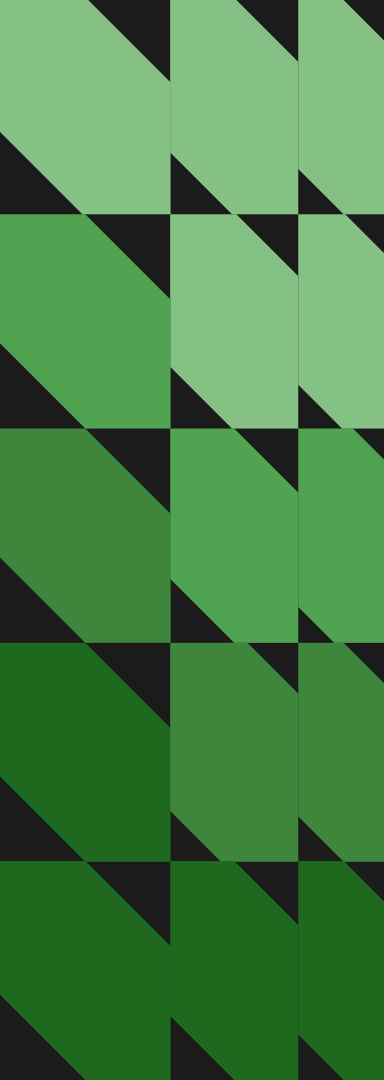


Camadas complementares

- Repository: Abstrai as fontes de dados (API, banco de dados local)
- Services: Classes que fazem as chamadas HTTP ou consultas no DB (Room, SQLite)
- Domain: Lógica de negócio específica separada da ViewModel.
- DTO: Converte objetos de dados da API para modelos de UI.

Clean Architecture





UDF vs MVVM

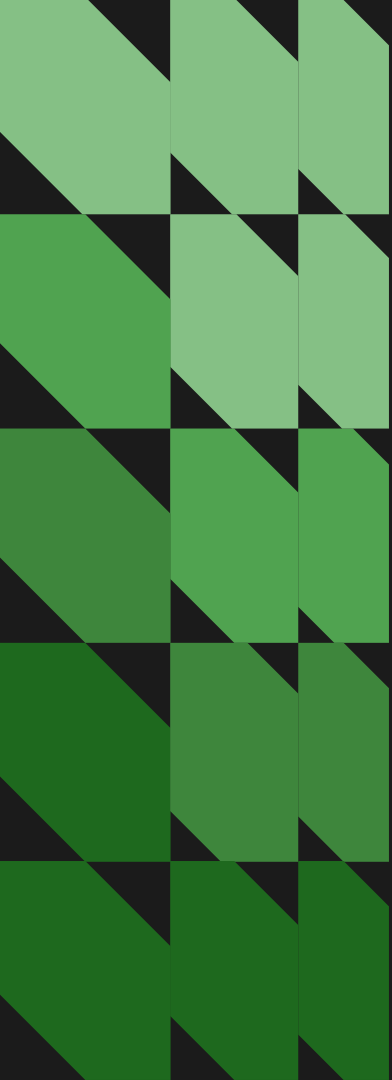
- UDF: É um padrão de fluxo de dados. Ele dita que o estado desce e os eventos sobem em um ciclo único.
- MVVM: É um padrão de arquitetura de software. Ele dita como você organiza o código em camadas funcionais.

Dividido em pelo menos três camadas: Model, ViewModel e View, além das camadas: Repository, Services, Domain e DTO.

Injeção de dependências

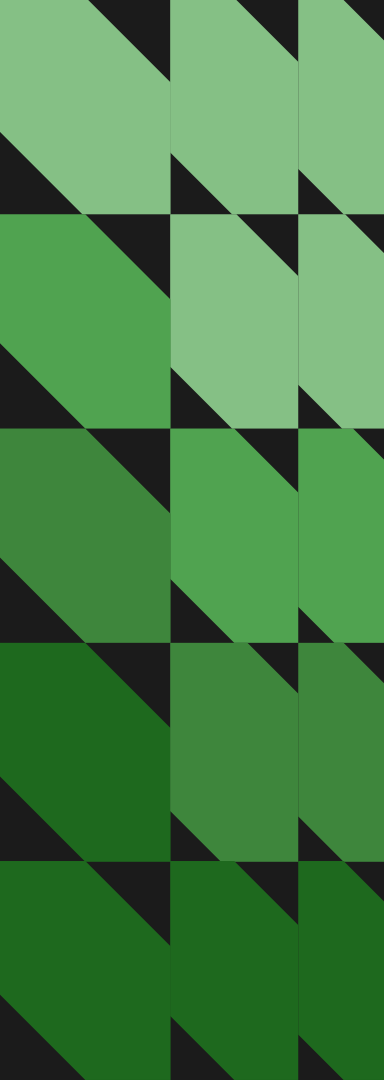
A injeção de dependências é um padrão de projeto que promove o desacoplamento de código, no qual uma classe recebe suas dependências de uma fonte externa, em vez de criá-las internamente.





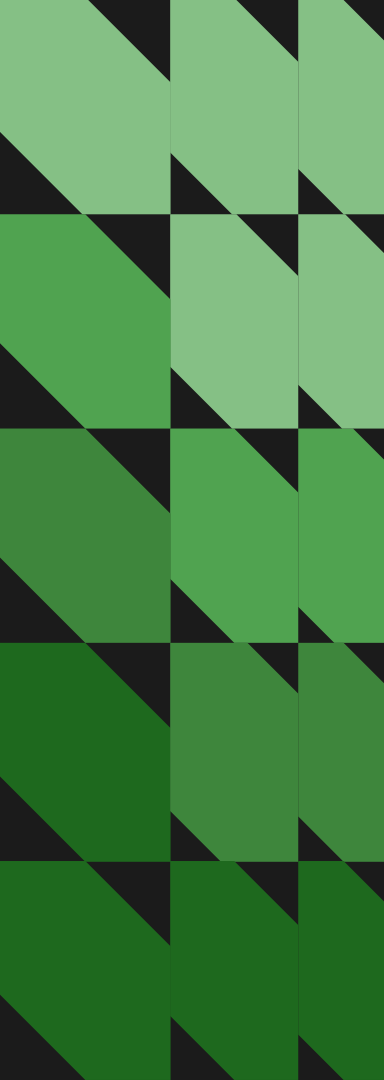
HILT

- O que é: Baseado no Dagger 2, mas simplificado para o Android.
- Vantagens: Integração nativa com Jetpack, ViewModel, Compose, e elimina muito código repetitivo (boilerplate) do Dagger.
- Ideal para: A maioria dos projetos Android modernos.
- **Importante! O Hilt é altamente recomendado pelo Google**



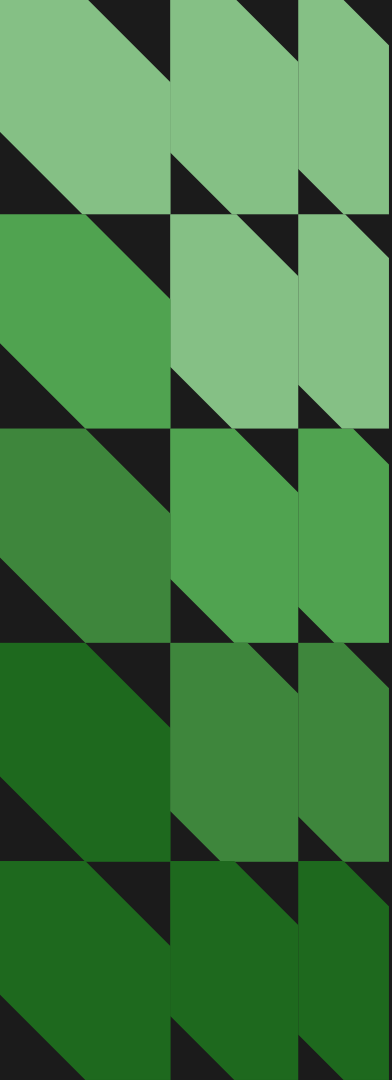
KOIN

- O que é: Framework leve, escrito em Kotlin, que funciona em tempo de execução (runtime).
- Vantagens: Muito fácil de aprender, sem geração de código e excelente para projetos Kotlin Multiplatform (KMP).
- Ideal para: Projetos menores, equipes que preferem simplicidade ou aplicações KMP.



DAGGER 2

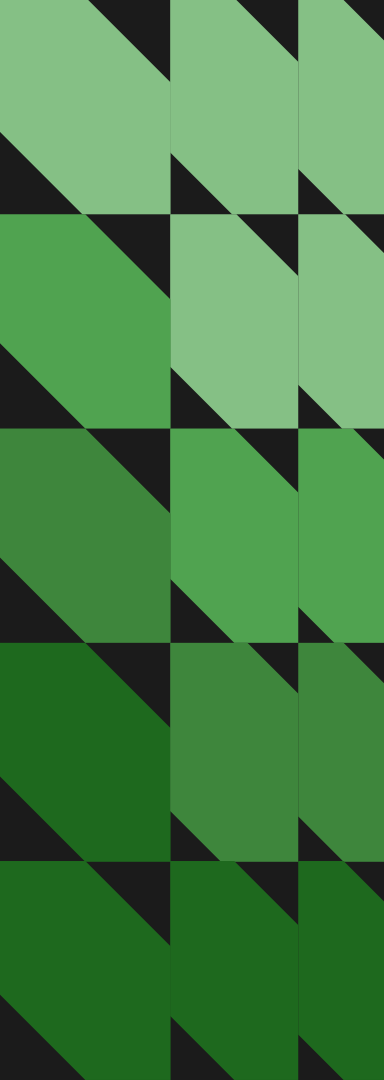
- O que é: O padrão da indústria, focado em compilação (compile-time).
- Vantagens: Performance alta e validação de dependências durante a compilação (erros aparecem antes de rodar o app).
- Desvantagens: Curva de aprendizado íngreme.
- Ideal para: Projetos muito grandes e complexos onde a performance é crítica.



Design Patterns

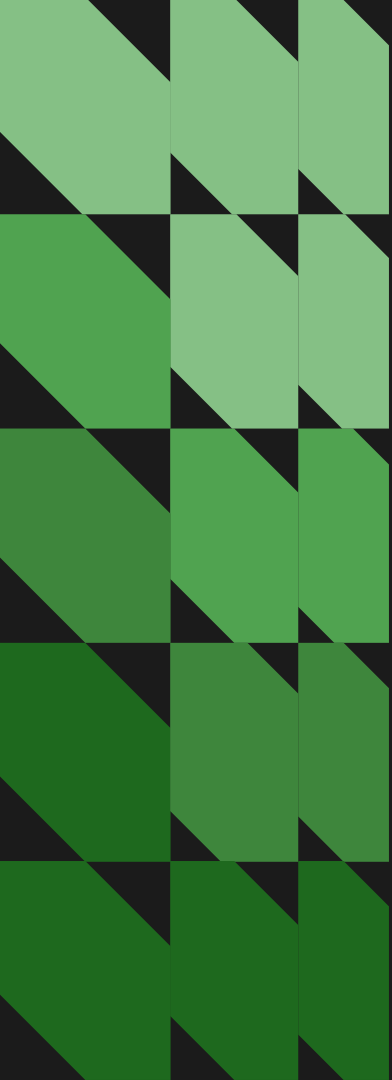
Design Patterns (padrões de projeto) são soluções reutilizáveis e testadas para problemas recorrentes no desenvolvimento de software. As principais características são:

- Reutilização;
- Padronização;
- Qualidade.



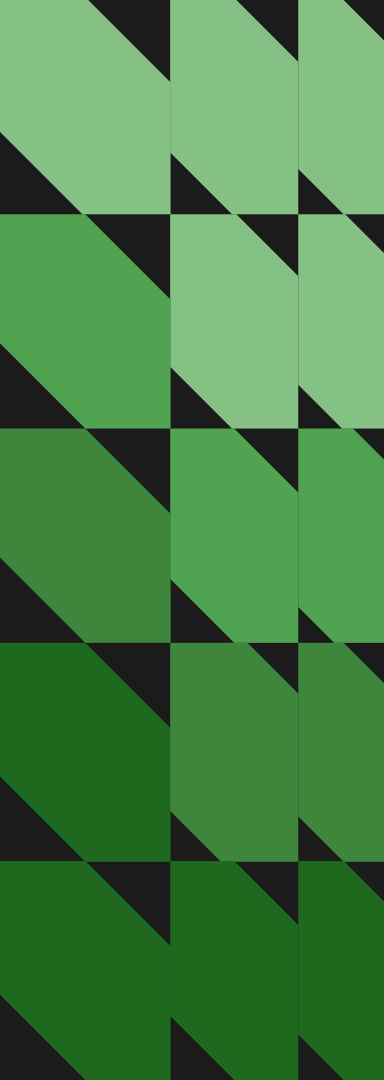
Tipos de Design Patterns

- Unidirectional Data Flow (UDF): Fluxo de dados
- Observer Pattern (LiveData ou StateFlow): Estrutura para exibir e alterar dados dinamicamente.
- Injeção de Dependências (HILT, KOIN ou DAGGER 2): Ao invés de instanciar objetos, podemos simplesmente usar o comando `@Inject`.



Tipos de Design Patterns

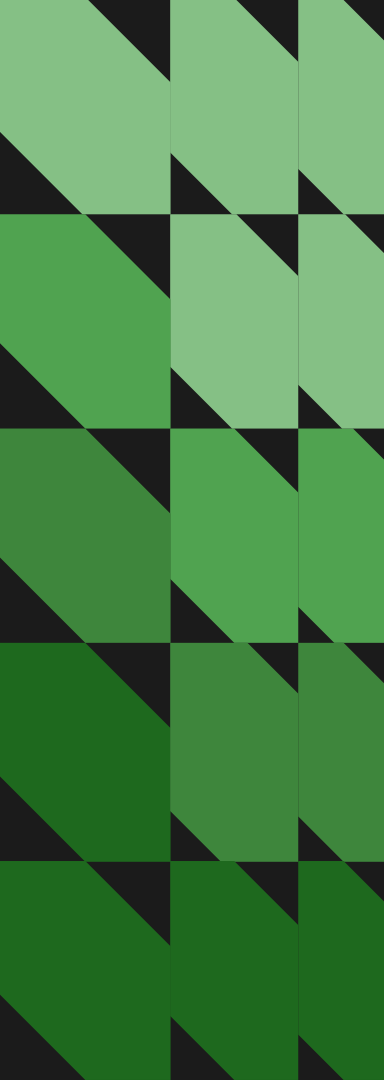
- Repository Pattern: Atua como um mediador entre diferentes fontes de dados (Banco de dados local, API remota, Cache).
- Factory Pattern: Muito utilizado para criar instâncias complexas, especialmente quando o sistema (Android).
- Hoisting (State Hoisting): Um padrão específico do Compose para tornar componentes "stateless" (sem estado interno).



Tipos de Design Patterns

- Singleton: Utilizado para garantir que certos componentes existam em apenas uma instância durante toda a vida do app.

Muito utilizado para termos um objeto de conexão com o banco SQLite.



MATERIAIS COMPLEMENTARES

- [GeeksForGeeks;](#)
- [Medium;](#)
- [Reintech;](#)
- [ProAndroidDev.](#)